

Hongbo Yu

+1 (346) 441-9593 | hy75@rice.edu | [linkedin.com/in/herbert-yu-rice](https://www.linkedin.com/in/herbert-yu-rice) | github.com/G1lollipop | Houston, TX

EDUCATION

Rice University

Master of Computer Science

Houston, TX

Aug. 2026 – Dec. 2027 (Expected)

Shenzhen University

B.E. in Computer Science and Technology

Shenzhen, China

Sep. 2021 – Jul. 2025

EXPERIENCE

Agent Development Intern

Autoagents

Mar. 2026 – Jul. 2026

Beijing, China

- Refactored linear agent workflows into a **LangGraph-based state-machine architecture**, adding **Redis-backed checkpointing** so long-running tasks could persist state, pause, and resume across asynchronous sessions.
- Improved tool-calling reliability by enforcing structured outputs with **Pydantic** and adding automatic retry and self-correction for malformed model responses, reducing workflow failures caused by API parsing errors.
- Built an evaluation and regression-testing workflow with **LangSmith** traces and golden test cases, allowing engineers to compare agent behavior and catch regressions before prompt and workflow changes reached production.
- Containerized backend services and vector databases and deployed them with **Kubernetes and Helm**; configured **HPA** and health probes to handle traffic spikes while maintaining **99.9% service availability**.

Backend Development Intern

Aqumon (Hongliang Research)

Mar. 2025 – Jul. 2025

Shenzhen, China

- Designed a **JWT + Redis** authentication system with access and refresh tokens, sliding expiration, and claims-based validation, allowing multi-tenant API requests to be verified at the gateway without repeated database lookups.
- Integrated **Stripe** with **RocketMQ** for asynchronous payment processing; used Redis idempotency keys, database uniqueness constraints, and explicit order-state transitions to prevent duplicate transactions and repeated payment handling.
- Diagnosed slow **MySQL** queries and redesigned query patterns, composite indexes, and covering indexes, reducing core API **p99 latency from 500ms to under 200ms**.

PROJECTS

FitCore – AI-Powered Fitness Platform | *Next.js, FastAPI, Supabase, pgvector*

Jan. 2026 – Present

Demo: fitcore-web-eight.vercel.app | **GitHub:** github.com/G1lollipop/fitcore

- Built a full-stack fitness platform in a monorepo with a **Next.js 16 / React 19** frontend and **FastAPI** backend; implemented typed Server Actions, middleware-based session refresh, and Supabase-backed authentication and persistence.
- Implemented real-time AI chat with **Server-Sent Events (SSE)** and **React Query** optimistic updates, streaming model responses while keeping time-to-first-token under **1 second**.
- Built a hybrid retrieval service combining semantic vector search and BM25 keyword search, followed by reranking and confidence-based filtering to improve the relevance of fitness and nutrition context returned to the application.
- Fine-tuned a lightweight language model with **LoRA/SFT** to convert free-form food descriptions into structured nutrition data, enforcing schema-constrained outputs with sub-**150ms** inference latency.

SKILLS

Languages: Java, Python, Go, C++, SQL, TypeScript

Backend & Frameworks: Spring Boot, FastAPI, Gin, Next.js, React

Databases & Middleware: MySQL, Redis, RocketMQ, Elasticsearch, Supabase, pgvector

Infrastructure & Tools: Docker, Kubernetes, Helm, Linux, Git, Maven

AI/ML: PyTorch, LangGraph, RAG, LoRA/SFT, Vector Search